UNIVERSIDAD DE GUADALAJARA

CENTRO UNIVERSITARIO DE CIENCIAS EXACTAS E INGENIERÍAS

Departamento de Innovación Basada en la Información y el Conocimiento



Ingeniería En Computación

PRÁCTICA 1

Uso de Hilos (Threads) en Python

Suma de un rango en paralelo y comparación con la suma secuencial

Nombre: GONZALEZ MARTINEZ ALONDRA GORETTI	**Código**: 218534681
Nombre de la clase: Programación paralela y recurrente	**Nombre del profesor:** Mtro. Zeus Cobián
Atributos de la competencia: Diseñar, implementar y probar algoritmos paralelos y concurrentes	**Actividad**: Práctica 1
Fecha de entrega: 09 de octubre del 2026	**Lenguaje**: Python 3.12

Bibliografía:

Silberschatz, A., Galvin, P. B., & Gagne, G. (2020). *Operating System Concepts*. John Wiley & Sons.

Tanenbaum, A. S., & Bos, H. (2015). *Modern Operating Systems*. Pearson Education.

Python Software Foundation. (2024). *threading — Thread-based parallelism*. Python 3 Documentation.

Recursos didácticos de la asignatura.

Contenido

Resumen . . . 3

1. Introducción . . . 3

Objetivos específicos . . . 4

2. Sustento teórico . . . 4

2.1 Procesos e hilos . . . 4

2.2 Paralelismo y concurrencia . . . 4

2.3 El módulo threading de Python . . . 5

2.4 Condición de carrera y sincronización . . . 5

2.5 Descomposición de datos . . . 5

2.6 El GIL de CPython . . . 5

2.7 Verificación y métricas . . . 6

3. Diseño de la solución . . . 6

3.1 División del rango entre los hilos . . . 6

3.2 Flujo de ejecución . . . 7

3.3 Pseudocódigo . . . 8

3.4 Funciones del programa . . . 8

4. Implementación paso a paso . . . 8

4.1 Ejemplo trazado: sumar del 1 al 12 con 3 hilos . . . 8

4.2 Los seis pasos del programa . . . 9

5. Código fuente . . . 10

Líneas clave del programa . . . 13

6. Pruebas y resultados . . . 13

6.1 Ejecución con 4 hilos . . . 13

6.2 Caso con división no exacta (N = 7) . . . 14

6.3 Pruebas de correctitud . . . 15

6.4 Análisis de tiempos . . . 15

6.5 Limitaciones . . . 16

7. Conclusión . . . 17

Referencias . . . 17

Anexo A. Código del benchmark . . . 18

Resumen

Este reporte documenta el diseño, la implementación y la validación de un programa en Python que suma los números enteros de un rango (del 1 al 1,000,000) repartiendo el trabajo entre *N* hilos configurables. Cada hilo calcula la suma de su subrango, el hilo principal espera con `join()` y acumula los resultados parciales, y el total se compara contra la suma secuencial y contra la fórmula cerrada de Gauss. Se incluyen el sustento teórico, un recorrido visual paso a paso con un ejemplo pequeño, el código completo con resaltado de sintaxis, las pruebas de correctitud y un análisis de tiempos que explica por qué, en CPython, más hilos no implican más velocidad.

500,000,500,000
resultado de la suma del 1 al 1,000,000

10 de 10
pruebas con resultados coincidentes

1 a 32
hilos evaluados en el benchmark

Tabla 0. Datos generales de la práctica.

Concepto	Detalle
Alumna	Gonzalez Martinez Alondra Goretti · código 218534681
Materia	Programación paralela y recurrente
Profesor	Mtro. Zeus Cobián
Actividad	Práctica 1 · Uso de hilos (threads) en Python (100 puntos)
Fecha de entrega	09 de octubre del 2026
Archivos entregados	informe en PDF, `suma_hilos.py` (programa), `benchmark_hilos.py` (mediciones)
Competencia	Diseñar, implementar y probar algoritmos paralelos y concurrentes

Cómo leer este reporte

Las secciones 3 y 4 muestran el proceso de forma visual (diagramas y un ejemplo trazado número por número); la sección 5 contiene el código completo; la sección 6 reúne los resultados medidos; y la 7 resume las conclusiones.

1. Introducción

Casi cualquier computadora actual tiene procesadores de varios núcleos, y el sistema operativo administra decenas de tareas a la vez. Para que un programa aproveche esa capacidad, o simplemente para que atienda varias actividades sin bloquearse, su trabajo debe dividirse en unidades más pequeñas de ejecución: los **hilos** (*threads*). Un hilo es la unidad básica de utilización del procesador; los hilos de un mismo proceso comparten memoria y recursos, por lo que son más ligeros que los procesos, pero exigen coordinación cuidadosa (Silberschatz et al., 2020).

Esta práctica aborda la creación y el manejo básico de hilos en Python con un problema sencillo y fácil de verificar: sumar los enteros de un rango. El rango se divide entre *N* hilos, cada uno suma su porción, y el hilo principal espera a que todos terminen para acumular los parciales. El resultado se contrasta con una

suma secuencial para comprobar que dividir el trabajo no altera el valor final.

> **Objetivo general**
> Comprender la creación y el manejo básico de hilos en Python mediante un programa que realice la suma de números en paralelo y compare su resultado con la suma secuencial.

Objetivos específicos

- Crear y lanzar un número configurable de hilos con el módulo `threading`.
- Dividir un rango de números de forma equitativa, incluso cuando la división no es exacta.
- Sincronizar al hilo principal con los hilos de trabajo mediante `join()` y combinar los resultados parciales.
- Verificar la correctitud comparando con la suma secuencial y con la fórmula de Gauss.
- Medir tiempos al variar N e interpretarlos a la luz del GIL de CPython.

2. Sustento teórico

2.1 Procesos e hilos

Un proceso es un programa en ejecución con su propio espacio de memoria. Un hilo es una ruta de ejecución dentro de un proceso: tiene su propio contador de programa, registros y pila, pero comparte código, datos y archivos abiertos con los demás hilos. Por compartir memoria, crear un hilo y comunicarlo con otros cuesta menos que hacerlo con procesos (Silberschatz et al., 2020; Tanenbaum y Bos, 2015).

Tabla 1. Comparación entre proceso e hilo.

Aspecto	Proceso	Hilo
Memoria	Espacio propio, aislado	Compartida con el resto de hilos del proceso
Costo de creación	Alto	Bajo
Comunicación	Mecanismos explícitos entre procesos (tuberías, colas)	Variables compartidas (requiere sincronizar)
Aislamiento ante fallos	Alto	Bajo: un error puede afectar a todo el proceso
En Python	`multiprocessing`	`threading`

2.2 Paralelismo y concurrencia

La **concurrencia** es la capacidad de manejar varias tareas durante un mismo periodo, intercalando su avance; el **paralelismo** es ejecutar varias tareas literalmente al mismo tiempo, lo cual requiere varios núcleos. Un programa con hilos es concurrente por diseño; será además paralelo solo si el entorno de ejecución permite que los hilos corran simultáneamente en distintos núcleos.

2.3 El módulo threading de Python

Tabla 2. Elementos de `threading` y su uso en la práctica.

Elemento	Descripción	Uso en la práctica
`Thread(target, args)`	Crea un hilo que ejecutará una función con argumentos dados.	Un hilo por cada subrango.
`start()`	Inicia la ejecución del hilo.	Se llama a los N hilos en un ciclo.
`join()`	Bloquea al hilo que lo llama hasta que el hilo termine.	El hilo principal espera a los N hilos.
`Lock`	Candado para proteger datos compartidos.	No se necesita: cada hilo escribe en su propia posición.

2.4 Condición de carrera y sincronización

Cuando varios hilos leen y modifican un mismo dato sin coordinación, el resultado puede depender del orden en que se intercalan sus instrucciones; esto se llama **condición de carrera**. Se evita con candados o diseñando el algoritmo para que cada hilo escriba solo en datos propios. En esta práctica se usa la segunda opción: cada hilo guarda su suma parcial en una posición exclusiva de una lista y solo el hilo principal las combina, después de los `join()`.

2.5 Descomposición de datos

El problema se resuelve con **descomposición de datos**: el mismo cálculo (sumar) se aplica a distintos subconjuntos de los datos. Si el rango tiene T números y hay N hilos, cada hilo recibe $T \,//\, N$ números y los primeros $T \bmod N$ hilos reciben uno adicional, de modo que ningún número se omite ni se cuenta dos veces.

2.6 El GIL de CPython

CPython, la implementación estándar de Python, incluye el *Global Interpreter Lock* (GIL): un candado que permite que solo un hilo ejecute bytecode de Python a la vez. Los hilos son muy útiles en tareas limitadas por entrada/salida (red, disco, usuario), pero **no aceleran** tareas limitadas por CPU, como sumar millones de enteros en un ciclo. Para paralelismo real de cálculo se usan procesos (`multiprocessing`) o extensiones nativas (Python Software Foundation, 2024; Beazley, 2010).

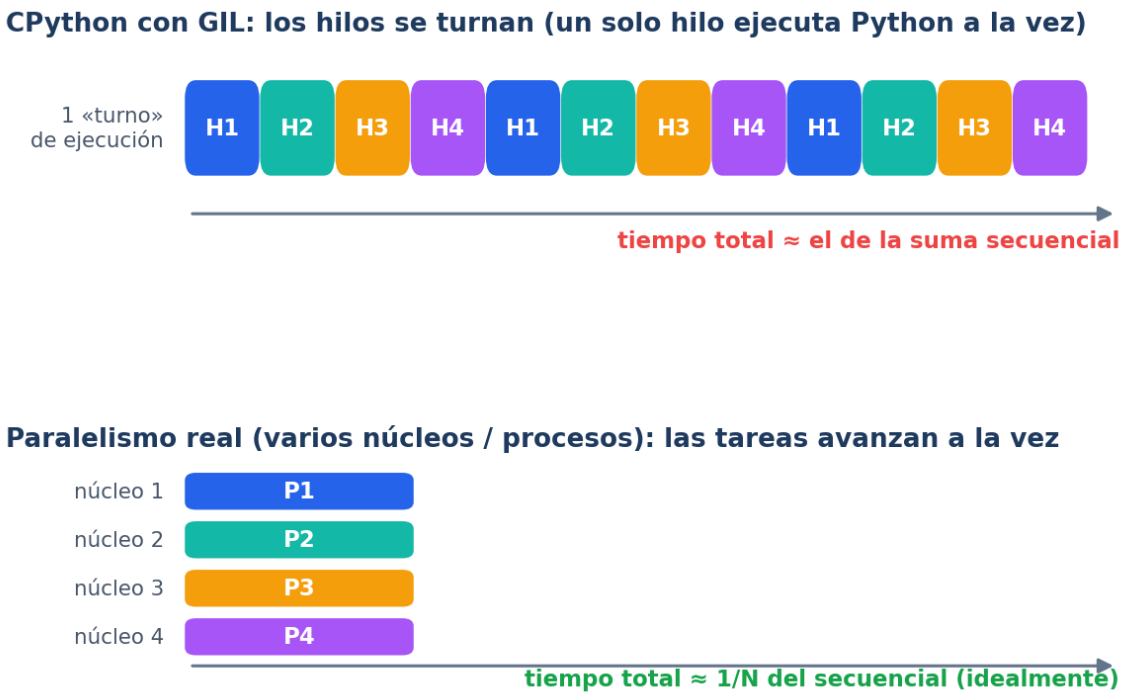


Figura 1. Comparación conceptual entre hilos con GIL (se turnan) y paralelismo real (simultáneo). *Fuente: elaboración propia.*

2.7 Verificación y métricas

Fórmula de Gauss. La suma de los enteros de a a b es $S = (a + b) \cdot (b - a + 1) / 2$. Para 1 a 1,000,000 resulta $(1 + 1{,}000{,}000) \cdot 1{,}000{,}000 / 2 = 500{,}000{,}500{,}000$, que sirve como comprobación independiente.

Aceleración y eficiencia. Con T_{sec} el tiempo secuencial y T_{par} el tiempo con N hilos, la aceleración es $S = T_{sec} / T_{par}$ y la eficiencia $E = S / N$. Un paralelismo ideal tendría $S = N$ y $E = 1$.

3. Diseño de la solución

3.1 División del rango entre los hilos

La función `dividir_rango` obtiene el tamaño base y el residuo con `divmod`. La Figura 2 muestra el resultado para $N = 4$ (división exacta) y $N = 7$ (el primer hilo toma un número extra).

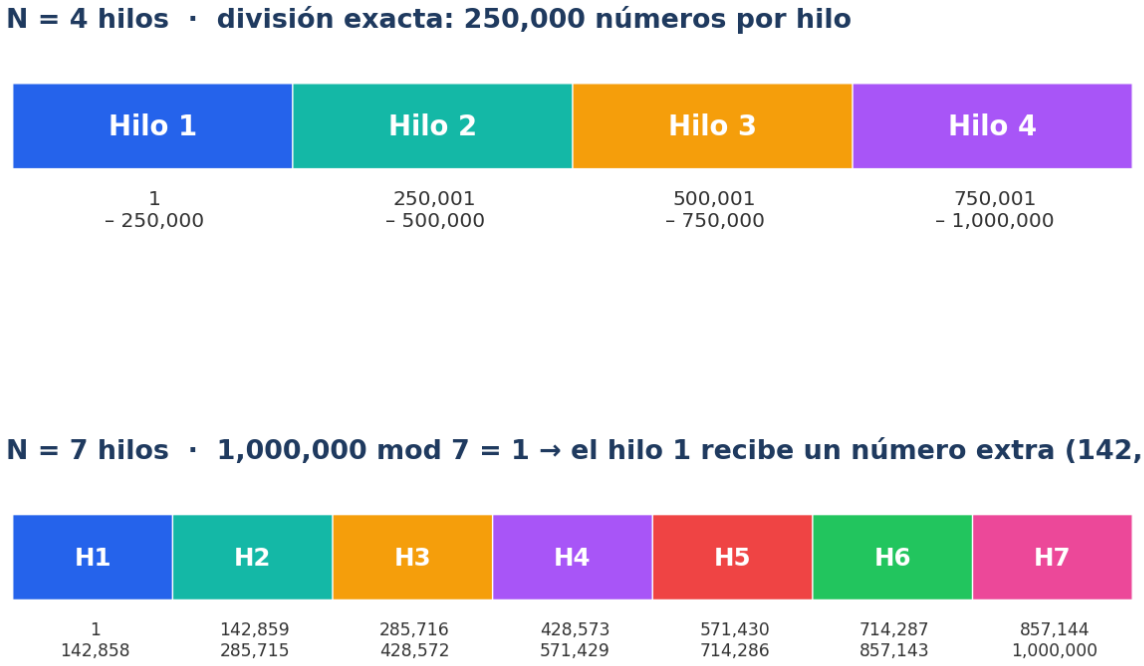


Figura 2. División del rango 1 a 1,000,000 entre 4 y entre 7 hilos; cada color es un hilo. *Fuente: elaboración propia.*

3.2 Flujo de ejecución

El hilo principal crea e inicia los hilos de trabajo, queda bloqueado en `join()` mientras ellos calculan y, cuando el último termina, acumula los parciales. Los hilos se lanzan escalonados porque cada `start()` ocurre uno tras otro.

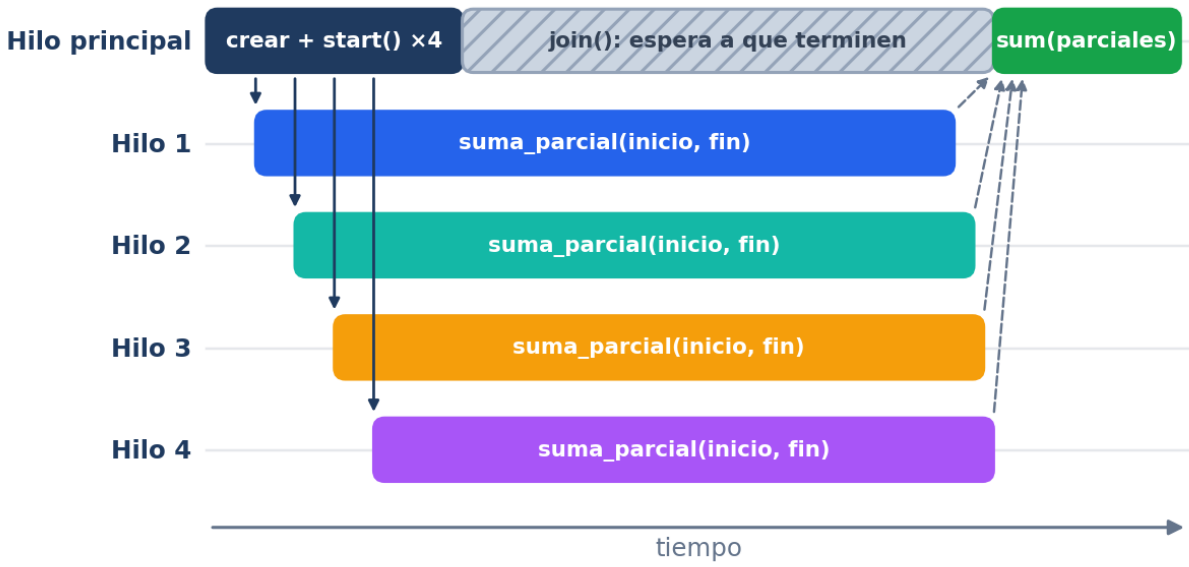


Figura 3. Línea de tiempo conceptual de la ejecución con 4 hilos (esquema, no medición). *Fuente: elaboración propia.*

3.3 Pseudocódigo

pseudocodigo.txt

```
ENTRADA: inicio, fin, N
rangos ← dividir(inicio, fin, N)            # divmod: tamaño base + residuo
PARA CADA subrango i EN rangos:
    crear hilo_i con suma_parcial(i, subrango_i)
    hilo_i.start()                          # se inicia el hilo
PARA CADA hilo_i:
    hilo_i.join()                           # el principal espera
total ← suma(resultados)                    # acumula los parciales
SI total == suma_secuencial == Gauss:
```

3.4 Funciones del programa

Tabla 3. Responsabilidad de cada función de `suma_hilos.py`.

Función	Responsabilidad	Salida
`leer_argumentos()`	Lee N y el rango (argumentos o teclado) y valida entradas inválidas.	Argumentos validados
`dividir_rango(...)`	Reparte el rango en N subrangos contiguos y casi iguales.	Lista de (inicio, fin)
`suma_parcial(...)`	Trabajo de cada hilo: suma su subrango.	Escribe en `resultados[i]`
`suma_con_hilos(...)`	Crea, inicia y espera a los hilos; acumula parciales.	(suma, rangos, parciales)
`suma_secuencial(...)`	Suma el rango sin hilos (referencia).	Entero
`main()`	Orquesta todo, mide tiempos e imprime la comparación.	Salida en consola

4. Implementación paso a paso

Esta sección sigue el proceso en el orden en que ocurre y muestra, junto a cada paso, el fragmento real del programa (los números a la izquierda son las líneas de `suma_hilos.py`). Primero se traza un ejemplo pequeño a mano para ver cada movimiento.

4.1 Ejemplo trazado: sumar del 1 al 12 con 3 hilos

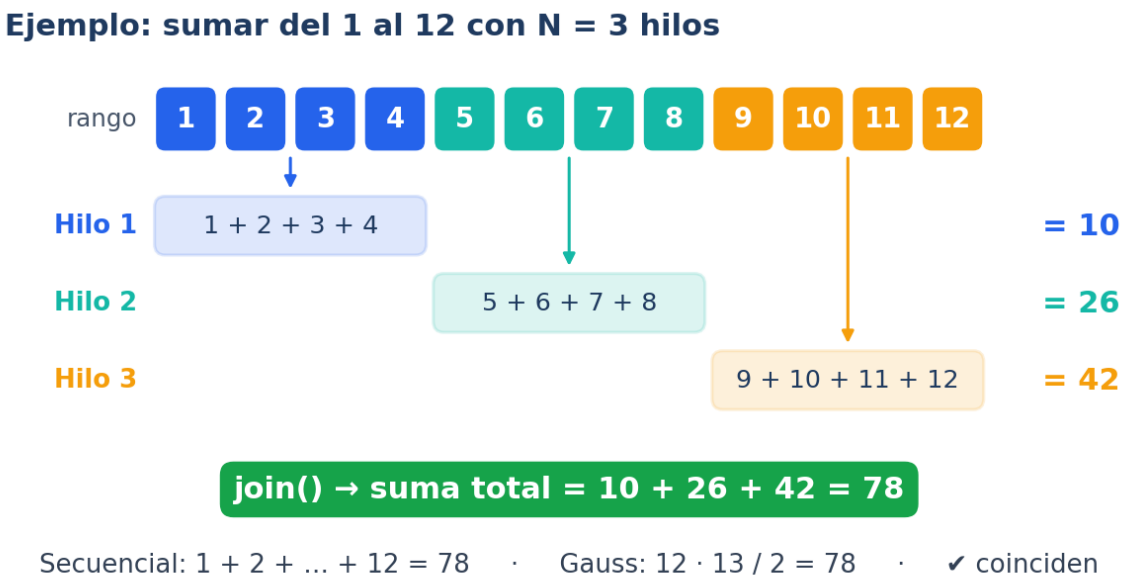


Figura 4. Cada hilo recibe 4 números, suma por su cuenta (10, 26 y 42) y el principal acumula 78. *Fuente: elaboración propia.*

Verificación del ejemplo

Suma con hilos: 10 + 26 + 42 = 78 · Suma secuencial: 78 · Gauss: 12 · 13 / 2 = 78. Los tres valores coinciden.

4.2 Los seis pasos del programa

1

Configurar N

El número de hilos y el rango llegan por la línea de comandos; si falta N, se pide por teclado. Se validan N menor que 1, rango invertido y más hilos que números.

```
77      parser.add_argument("-n", "--hilos", type=int, help="número de hilos (N)")
```

```
98      args = leer_argumentos()
```

2

Dividir el rango

Con divmod se obtienen el tamaño base y el residuo; los primeros hilos reciben un número extra para no perder ningún valor.

```
27      base, residuo = divmod(total, n_hilos)
```

```
31          tamano = base + (1 if i < residuo else 0)
32          rangos.append((actual, actual + tamano - 1))
33          actual += tamano
```

3 Crear e iniciar los hilos

Por cada subrango se crea un Thread cuyo objetivo es suma_parcial y se inicia con start(); todos quedan en ejecución concurrente.

```
56          hilo = threading.Thread(target=suma_parcial, args=(i, ini, fi, resultados))
57          hilos.append(hilo)
58          hilo.start()
```

4 Cálculo parcial en cada hilo

Cada hilo acumula la suma de su subrango y la guarda en su propia posición de la lista de resultados, sin tocar la de los demás (sin condición de carrera).

```
43      acumulado = 0
44      for numero in range(inicio, fin + 1):
45          acumulado += numero
46      resultados[indice] = acumulado
```

5 Esperar con join()

El hilo principal recorre la lista de hilos y llama a join() en cada uno: no avanza hasta que todos hayan terminado.

```
61      for hilo in hilos:
62          hilo.join()
```

6 Acumular y comparar

El principal suma los parciales, calcula la suma secuencial y la fórmula de Gauss, y compara los tres valores.

```
64      return sum(resultados), rangos, resultados
```

```
125     if total_hilos == total_sec == total_formula:
126         print("RESULTADO: los tres valores COINCIDEN.")
127     else:
128         print("RESULTADO: los valores NO coinciden (revisar el programa).")
```

5. Código fuente

Listado completo de `suma_hilos.py` con resaltado de sintaxis. Para ejecutarlo:

terminal · cómo ejecutar

```
$ python suma_hilos.py            # pide N por teclado (rango 1 a 1,000,000)
$ python suma_hilos.py -n 4         # 4 hilos
$ python suma_hilos.py -n 8 --fin 5000000   # 8 hilos, rango 1 a 5,000,000
```

suma_hilos.py · Python 3

```python
"""Práctica 1: Uso de hilos (threads) en Python.
Alumna: Alondra Goretti González Martínez (218534681)
Materia: Programación paralela y recurrente - Mtro. Zeus Cobián

Suma de los números de un rango [inicio, fin] de dos formas:
  1) En paralelo, dividiendo el rango entre N hilos.
  2) De forma secuencial (sin hilos).
Después compara ambos resultados.

Uso:
    python suma_hilos.py                    # pide N por teclado (rango 1..1,000,000)
    python suma_hilos.py -n 4               # 4 hilos
    python suma_hilos.py -n 8 --fin 5000000 # 8 hilos, rango 1..5,000,000
"""
import argparse
import threading
import time


def dividir_rango(inicio, fin, n_hilos):
    """Divide [inicio, fin] en n_hilos subrangos contiguos y casi iguales.

    Si la cantidad de números no es divisible entre n_hilos, los primeros
    'residuo' hilos reciben un número extra, así no se pierde ningún valor.
    """
    total = fin - inicio + 1
    base, residuo = divmod(total, n_hilos)
    rangos = []
    actual = inicio
    for i in range(n_hilos):
        tamano = base + (1 if i < residuo else 0)
        rangos.append((actual, actual + tamano - 1))
        actual += tamano
    return rangos


def suma_parcial(indice, inicio, fin, resultados):
    """Trabajo de cada hilo: suma los enteros de [inicio, fin].

    Cada hilo escribe solo en su propia posición de 'resultados', por lo que
    no hay condición de carrera y no se necesita un candado (Lock).
    """
    acumulado = 0
    for numero in range(inicio, fin + 1):
        acumulado += numero
    resultados[indice] = acumulado


def suma_con_hilos(inicio, fin, n_hilos):
    """Suma el rango usando n_hilos hilos. Devuelve (suma, rangos, parciales)."""
    rangos = dividir_rango(inicio, fin, n_hilos)
    resultados = [0] * n_hilos
    hilos = []

    for i, (ini, fi) in enumerate(rangos):
        hilo = threading.Thread(target=suma_parcial, args=(i, ini, fi, resultados))
        hilos.append(hilo)
        hilo.start()

    # El hilo principal espera a que todos terminen
    for hilo in hilos:
        hilo.join()

    return sum(resultados), rangos, resultados


```

suma_hilos.py · Python 3

```python
def suma_secuencial(inicio, fin):
    """Suma el rango en un solo flujo de ejecución (sin hilos)."""
    acumulado = 0
    for numero in range(inicio, fin + 1):
        acumulado += numero
    return acumulado


def leer_argumentos():
    parser = argparse.ArgumentParser(description="Suma de un rango con hilos vs secuencial")
    parser.add_argument("-n", "--hilos", type=int, help="número de hilos (N)")
    parser.add_argument("--inicio", type=int, default=1, help="inicio del rango (default 1)")
    parser.add_argument("--fin", type=int, default=1_000_000, help="fin del rango (default 1,000,000)")
    args = parser.parse_args()

    if args.hilos is None:
        try:
            args.hilos = int(input("Número de hilos (N): "))
        except ValueError:
            parser.error("N debe ser un entero")

    if args.hilos < 1:
        parser.error("N debe ser mayor o igual a 1")
    if args.fin < args.inicio:
        parser.error("--fin debe ser mayor o igual a --inicio")
    if args.hilos > args.fin - args.inicio + 1:
        parser.error("N no puede ser mayor que la cantidad de números del rango")
    return args


def main():
    args = leer_argumentos()

    print(f"Rango: {args.inicio:,} a {args.fin:,}  |  Hilos (N): {args.hilos}\n")

    # --- Suma con hilos ---
    t0 = time.perf_counter()
    total_hilos, rangos, parciales = suma_con_hilos(args.inicio, args.fin, args.hilos)
    t_hilos = time.perf_counter() - t0

    for i, ((ini, fi), parcial) in enumerate(zip(rangos, parciales), start=1):
        print(f"Hilo {i:>2}: [{ini:>9,} - {fi:>9,}]  suma parcial = {parcial:,}")

    # --- Suma secuencial ---
    t0 = time.perf_counter()
    total_sec = suma_secuencial(args.inicio, args.fin)
    t_sec = time.perf_counter() - t0

    # --- Verificación extra con la fórmula de Gauss ---
    n = args.fin - args.inicio + 1
    total_formula = (args.inicio + args.fin) * n // 2

    print("\n" + "=" * 60)
    print(f"Suma con {args.hilos} hilo(s): {total_hilos:,}   ({t_hilos:.4f} s)")
    print(f"Suma secuencial   : {total_sec:,}   ({t_sec:.4f} s)")
    print(f"Fórmula de Gauss  : {total_formula:,}")
    print("=" * 60)

    if total_hilos == total_sec == total_formula:
        print("RESULTADO: los tres valores COINCIDEN.")
    else:
        print("RESULTADO: los valores NO coinciden (revisar el programa).")


if __name__ == "__main__":
    main()
```

Líneas clave del programa

Tabla 4. Ubicación de los elementos principales en el listado.

Elemento	Línea	Qué hace
División del rango	20–34	Calcula tamaño base y residuo; arma los subrangos.
Trabajo del hilo	37–46	Suma el subrango y guarda el parcial en su posición.
Creación del hilo	56	Un `Thread` por subrango.
Inicio del hilo	58	Pone el hilo en ejecución.
Espera (join)	62	El principal espera a que termine cada hilo.
Acumulación	64	Suma los parciales de todos los hilos.
Suma secuencial	67	Referencia sin hilos.
Comparación	125	Verifica hilos = secuencial = Gauss.

6. Pruebas y resultados

6.1 Ejecución con 4 hilos

La Figura 5 muestra la salida real del programa con $N = 4$. Cada hilo recibió 250,000 números y devolvió una suma parcial distinta, porque los números más altos pesan más.

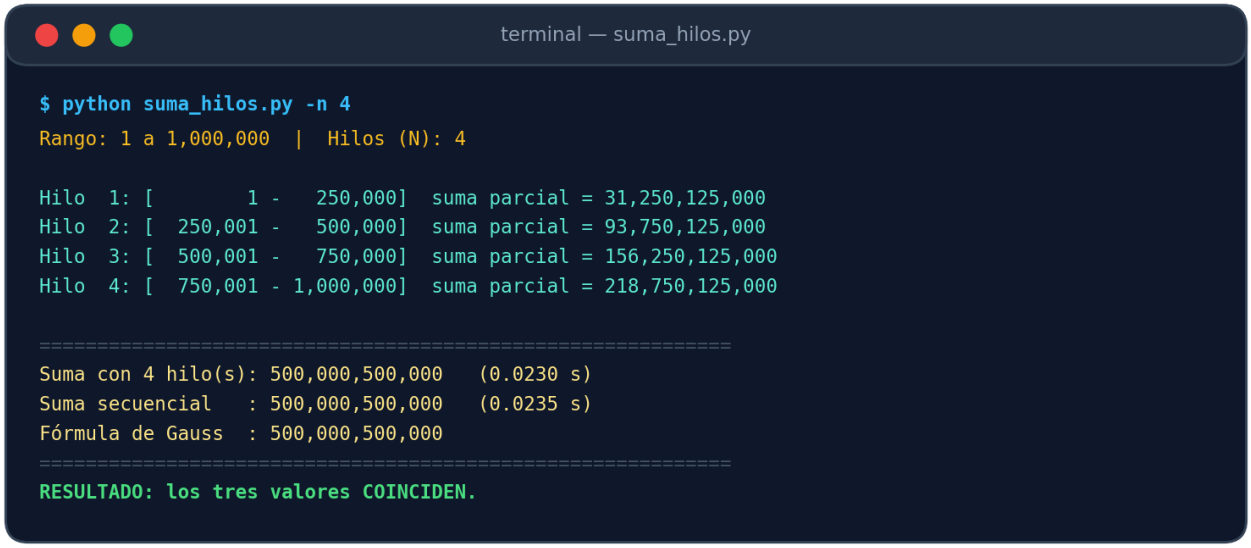


Figura 5. Salida en consola de `python suma_hilos.py -n 4`. *Fuente: elaboración propia.*

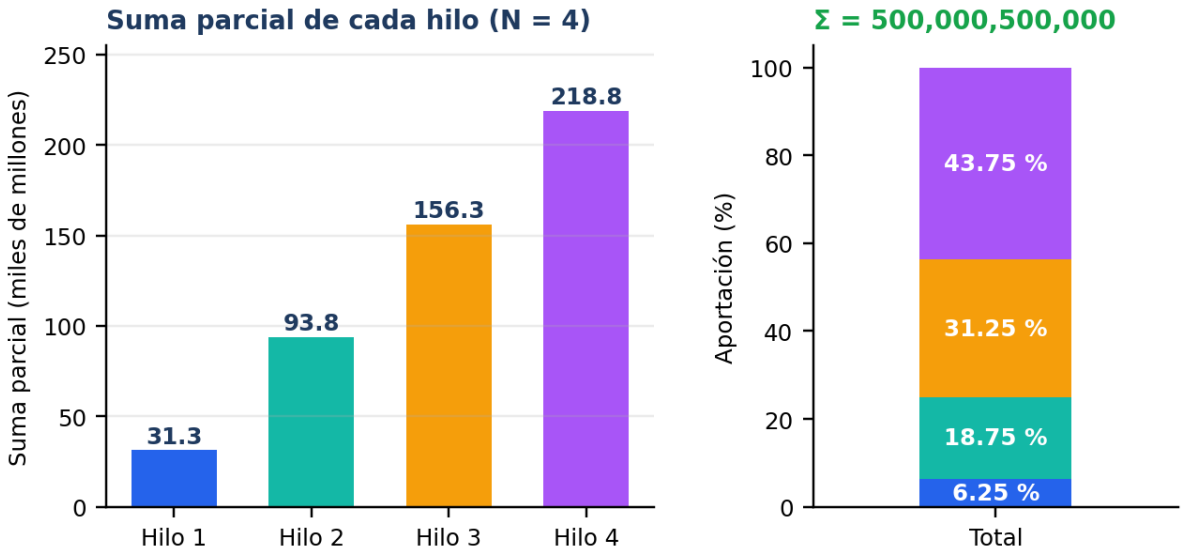


Figura 6. Sumas parciales por hilo (izquierda) y su aportación porcentual al total (derecha). *Fuente: elaboración propia.*

6.2 Caso con división no exacta (N = 7)

Como 1,000,000 no es divisible entre 7, el primer hilo recibe un número adicional (142,858 frente a 142,857). El total sigue siendo correcto.

```
terminal — suma_hilos.py

$ python suma_hilos.py -n 7
Rango: 1 a 1,000,000  |  Hilos (N): 7

Hilo  1: [        1 -   142,858]  suma parcial = 10,204,275,511
Hilo  2: [  142,859 -   285,715]  suma parcial = 30,612,397,959
Hilo  3: [  285,716 -   428,572]  suma parcial = 51,020,520,408
Hilo  4: [  428,573 -   571,429]  suma parcial = 71,428,642,857
Hilo  5: [  571,430 -   714,286]  suma parcial = 91,836,765,306
Hilo  6: [  714,287 -   857,143]  suma parcial = 112,244,887,755
Hilo  7: [  857,144 - 1,000,000]  suma parcial = 132,653,010,204

============================================================
Suma con 7 hilo(s): 500,000,500,000   (0.0230 s)
Suma secuencial   : 500,000,500,000   (0.0232 s)
Fórmula de Gauss  : 500,000,500,000
============================================================
RESULTADO: los tres valores COINCIDEN.
```

Figura 7. Salida en consola de `python suma_hilos.py -n 7`. *Fuente: elaboración propia.*

6.3 Pruebas de correctitud

Tabla 5. Resultados de las pruebas: hilos, secuencial y fórmula de Gauss.

#	N	Rango	Suma con hilos	Secuencial	Gauss	Resultado
1	1	1 – 1,000,000	500,000,500,000	500,000,500,000	500,000,500,000	✔ Coincide
2	2	1 – 1,000,000	500,000,500,000	500,000,500,000	500,000,500,000	✔ Coincide
3	3	1 – 1,000,000	500,000,500,000	500,000,500,000	500,000,500,000	✔ Coincide
4	4	1 – 1,000,000	500,000,500,000	500,000,500,000	500,000,500,000	✔ Coincide
5	7	1 – 1,000,000	500,000,500,000	500,000,500,000	500,000,500,000	✔ Coincide
6	8	1 – 1,000,000	500,000,500,000	500,000,500,000	500,000,500,000	✔ Coincide
7	16	1 – 1,000,000	500,000,500,000	500,000,500,000	500,000,500,000	✔ Coincide
8	32	1 – 1,000,000	500,000,500,000	500,000,500,000	500,000,500,000	✔ Coincide
9	5	1 – 1,000,003	500,003,500,006	500,003,500,006	500,003,500,006	✔ Coincide
10	10	50 – 1,000	499,275	499,275	499,275	✔ Coincide

Resultado

En las 10 pruebas coincidieron los tres valores, incluidos los casos con división no exacta (N = 3, 7 y 5 con 1,000,003 números) y con un rango que no empieza en 1.

6.4 Análisis de tiempos

Para medir con más confianza se ejecutó `benchmark_hilos.py`, que repite cada configuración 7 veces y reporta la mediana, el mínimo y el máximo.

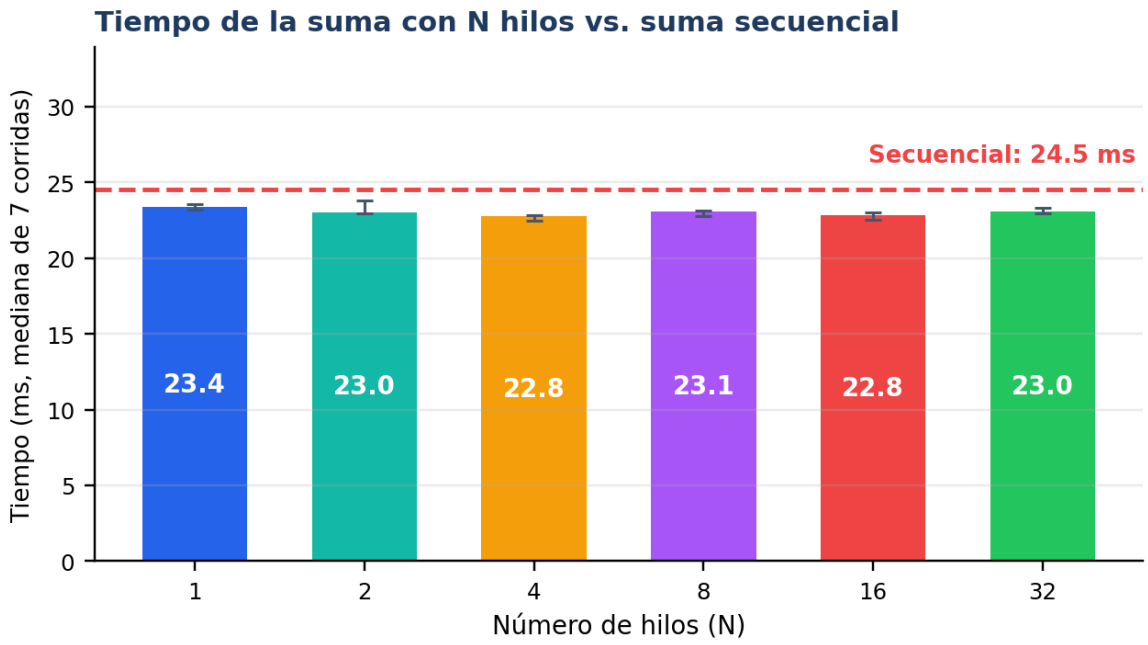


Figura 8. Mediana de 7 corridas por configuración; la línea roja es la suma secuencial y las barras de error indican mínimo y máximo. *Fuente: elaboración propia.*

Tabla 6. Tiempos medidos, aceleración y eficiencia (rango 1 a 1,000,000).

Configuración	Mediana (ms)	Mín. – Máx. (ms)	Aceleración S	Eficiencia E
Secuencial	24.5	23.7 – 24.8	1.00	—
1 hilo(s)	23.4	23.2 – 23.6	1.05	104.9 %
2 hilo(s)	23.0	22.9 – 23.8	1.06	53.2 %
4 hilo(s)	22.8	22.4 – 22.8	1.08	26.9 %
8 hilo(s)	23.1	22.8 – 23.1	1.06	13.3 %
16 hilo(s)	22.8	22.5 – 23.0	1.07	6.7 %
32 hilo(s)	23.0	22.9 – 23.3	1.06	3.3 %

Nota: aceleraciones ligeramente mayores a 1 (por ejemplo, 1.05) corresponden al ruido de medición, no a una mejora real; por eso la eficiencia con 1 hilo puede superar el 100 %.

Hallazgo principal

Los tiempos con 1 a 32 hilos son prácticamente iguales entre sí y al secuencial (alrededor de 23 ms). La aceleración queda cerca de 1.0 y la eficiencia cae conforme crece N. No es un error del programa: es el efecto del GIL, que deja ejecutar un solo hilo de Python a la vez.

La suma es una tarea limitada por CPU; mientras el GIL esté activo, los hilos se turnan en lugar de avanzar a la par. Crear hasta 32 hilos cuesta muy poco frente al trabajo total, por lo que tampoco hay un empeoramiento apreciable. Las pequeñas diferencias entre configuraciones están dentro del ruido de medición. En estos resultados, los hilos demuestran la **estructura** de un programa paralelo (división, lanzamiento, sincronización y acumulación) y la **correctitud**, pero no una aceleración.

Para obtener aceleración real

Se podría conservar la misma lógica de división del rango y sustituir los hilos por procesos con `multiprocessing`, que no comparten el GIL. La mejora dependería del número de núcleos del equipo (el entorno de pruebas de este reporte tenía un solo núcleo, por lo que no habría permitido demostrarla).

6.5 Limitaciones

- Los tiempos absolutos dependen del equipo y del momento; la conclusión sobre el GIL no depende de ellos.
- Con rangos muy pequeños, el costo de crear hilos puede superar al de la suma misma.
- Los tiempos de este reporte se midieron en un entorno de pruebas; conviene repetir `benchmark_hilos.py` en otro equipo y comparar los resultados.

7. Conclusión

Correctitud

El total 500,000,500,000 coincidió con la suma secuencial y con Gauss en todas las pruebas, incluso con división no exacta.

Sincronización

join() garantizó que el principal acumulara solo cuando todos los hilos terminaron; sin candados, porque no se comparte ninguna posición.

Rendimiento

Más hilos no redujeron el tiempo: el GIL de CPython impide la ejecución simultánea de código Python de cálculo.

Siguiente paso

Aplicar la misma división con multiprocessing, o usar hilos en tareas de entrada/salida, donde sí aportan.

La práctica permitió crear y administrar hilos en Python: se configuró el número de hilos *N*, se dividió el rango de 1 a 1,000,000 en subrangos casi iguales, cada hilo calculó su suma parcial y el hilo principal esperó con `join()` para acumular los resultados. El total coincidió con la suma secuencial y con la fórmula de Gauss en todas las pruebas, lo que confirma que la descomposición de datos fue correcta.

También se observó que usar más hilos no redujo el tiempo de ejecución. Esto no indica un error, sino una característica de CPython: el GIL impide que los hilos ejecuten código Python de cálculo de manera simultánea. Comprender la diferencia entre *concurrencia* (organizar tareas) y *paralelismo* (ejecutarlas al mismo tiempo) es importante al diseñar algoritmos paralelos: los hilos de Python son útiles cuando el programa espera entrada/salida, mientras que, para acelerar cálculo intensivo, conviene usar procesos u otras herramientas. En mi opinión, la práctica deja claros el patrón de división, sincronización y verificación que servirá de base para los temas siguientes de la materia.

Referencias

Beazley, D. (2010). *Understanding the Python GIL*. PyCon 2010.

Python Software Foundation. (2024). *threading — Thread-based parallelism*. Python 3 Documentation. https://docs.python.org/3/library/threading.html

Silberschatz, A., Galvin, P. B. y Gagne, G. (2020). *Operating System Concepts* (10.ª ed.). John Wiley & Sons.

Tanenbaum, A. S. y Bos, H. (2015). *Modern Operating Systems* (4.ª ed.). Pearson Education.

Anexo A. Código del benchmark

Script complementario que repite cada configuración 7 veces y calcula la mediana, el mínimo y el máximo del tiempo (importa las funciones de `suma_hilos.py`).

benchmark_hilos.py · Python 3

```python
"""Benchmark complementario: mide el tiempo (mediana de varias repeticiones)
de la suma con N hilos frente a la suma secuencial.
Uso: python benchmark_hilos.py
"""
import json
import statistics
import time

from suma_hilos import suma_con_hilos, suma_secuencial

INICIO, FIN = 1, 1_000_000
REPETICIONES = 7
VALORES_N = [1, 2, 4, 8, 16, 32]


def medir(funcion, *args):
    tiempos = []
    for _ in range(REPETICIONES):
        t0 = time.perf_counter()
        funcion(*args)
        tiempos.append(time.perf_counter() - t0)
    return statistics.median(tiempos), min(tiempos), max(tiempos)


if __name__ == "__main__":
    resultados = {"secuencial": medir(suma_secuencial, INICIO, FIN), "hilos": {}}
    for n in VALORES_N:
        resultados["hilos"][n] = medir(suma_con_hilos, INICIO, FIN, n)
        print(f"N={n:>2}  mediana={resultados['hilos'][n][0]:.4f} s")
    print(f"Secuencial  mediana={resultados['secuencial'][0]:.4f} s")
    with open("benchmark.json", "w") as f:
        json.dump(resultados, f)
```